

Deep Learning — Assignment 1

First assignment for the 2023 Deep Learning course (NWI-IMC070) of the Radboud University.

Names: Vojko Pjanovic (s1084416); Carmen Gascó Gálvez (s1098566)

Group: 10

Instructions:

- Fill in your names and the name of your group.
- Answer the questions and complete the code where necessary.
- Keep your answers brief, one or two sentences is usually enough.
- Re-run the whole notebook before you submit your work.
- Save the notebook as a PDF and submit that in Brightspace together with the `.ipynb` notebook file.
- The easiest way to make a PDF of your notebook is via File > Print Preview and then use your browser's print option to print to PDF.

Objectives

In this assignment you will

1. Experiment with gradient descent optimization;
2. Derive and implement gradients for binary cross-entropy loss, the sigmoid function and a linear layer;
3. Test your gradient implementations with the finite difference method;
4. Use these components to implement and train a simple neural network.

```
In [ ]: %matplotlib inline
import numpy as np
import scipy.optimize
import sklearn.datasets
import matplotlib.pyplot as plt

np.set_printoptions(suppress=True, precision=6, linewidth=200)
plt.style.use('ggplot')
```

1.1 Gradient descent optimization (6 points)

Consider the following function with two parameters and its derivatives:

$$f(x, y) = x^2 + y^2 + x(y + 2) + \cos(3x) \quad (1)$$

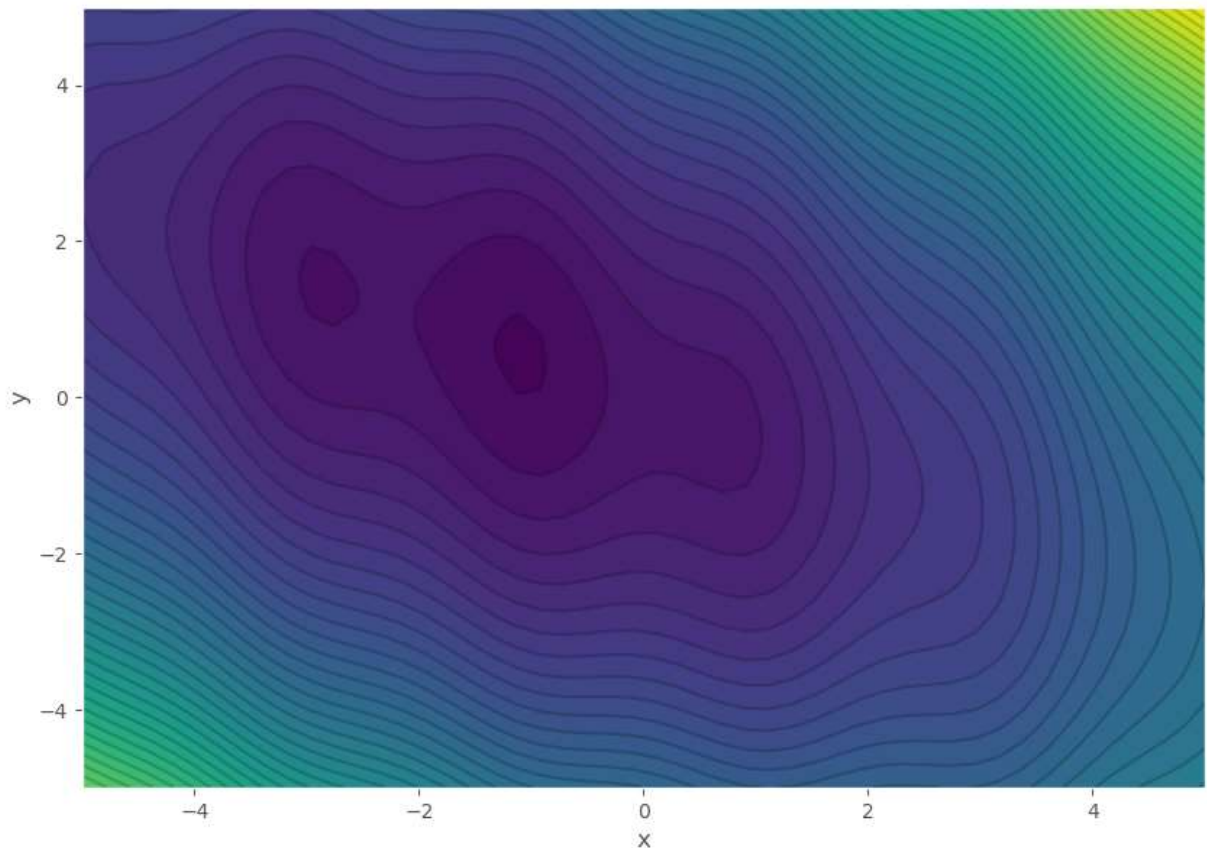
$$\frac{\partial f}{\partial x} = 2x - 3\sin(3x) + y + 2 \quad (2)$$

$$\frac{\partial f}{\partial y} = x + 2y \quad (3)$$

```
In [ ]: def f(x, y):  
        return x ** 2 + y ** 2 + x * (y + 2) + np.cos(3 * x)  
def grad_x_f(x, y):  
    return 2 * x - 3 * np.sin(3 * x) + y + 2  
def grad_y_f(x, y):  
    return x + 2 * y
```

A plot of the function shows that it has multiple local minima:

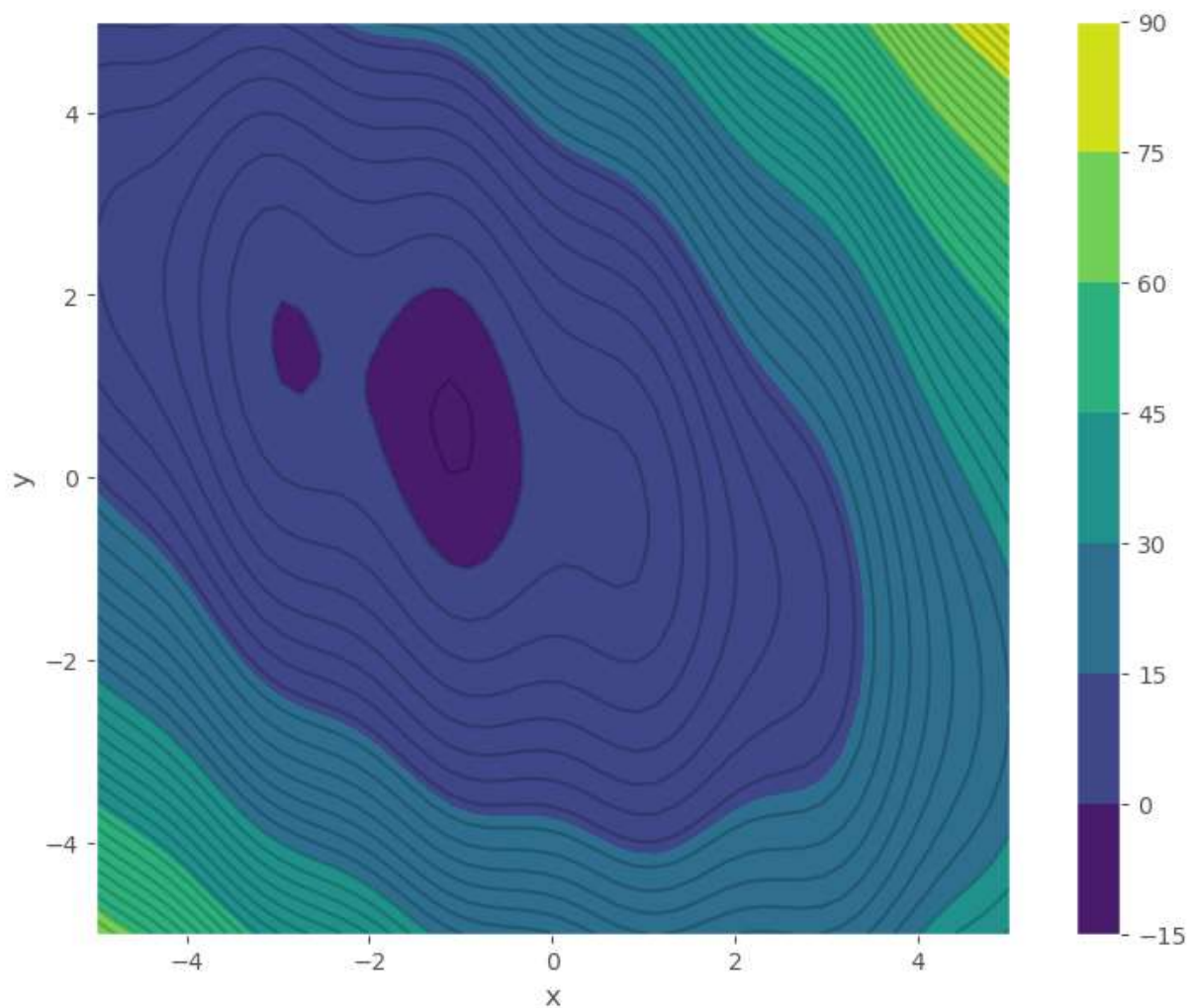
```
In [ ]: def plot_f_contours():  
    xx, yy = np.meshgrid(np.linspace(-5, 5), np.linspace(-5, 5))  
    zz = f(xx, yy)  
    plt.contourf(xx, yy, zz, 50)  
    plt.contour(xx, yy, zz, 50, alpha=0.2, colors='black', linestyle='solid')  
    plt.xlabel('x')  
    plt.ylabel('y')  
  
plt.figure(figsize=(10, 7))  
plot_f_contours()
```



```
In [ ]: # CODE FOR THE SIDE BAR
```

```
def plot_f_contours():
    xx, yy = np.meshgrid(np.linspace(-5, 5), np.linspace(-5, 5))
    zz = f(xx, yy)
    plt.contourf(xx, yy, zz, 50)
    plt.contour(xx, yy, zz, 50, alpha=0.2, colors='black', linestyle='solid')
    plt.xlabel('x')
    plt.ylabel('y')
    ax.set_aspect('equal')
    cf = ax.contourf(xx,yy,zz)
    fig.colorbar(cf, ax=ax)

fig, ax = plt.subplots(figsize=(10,7))
plot_f_contours()
```



Implement gradient descent

We would like to find the minimum of this function using gradient descent.

(a) Implement the gradient descent updates for x and y in the function below:(1 point)

```
In [ ]: def optimize_f(x, y, step_size, steps):
        # keep track of the parameters we tried so far
        x_hist, y_hist = [x], [y]

        # run gradient descent for the number of steps
        for step in range(steps):
            # compute the gradients at the current point
            dx = grad_x_f(x, y)
            dy = grad_y_f(x, y)

            # apply the gradient descent updates to x and y
            x -= step_size * dx
            y -= step_size * dy

            # store the new parameters
            x_hist.append(x)
            y_hist.append(y)

        return x, y, f(x, y), x_hist, y_hist
```

```
In [ ]: # The following assert statements check that your implementation behaves sensibly
        # Use it to get a hint only if you are stuck.
        assert optimize_f(3, 2, 0.1, 1)[0] != 3, "Hint: you are not changing `x`"
        assert optimize_f(3, 2, 0.1, 1)[2] < f(3, 2), "Hint: the function value is increasing"
        assert abs(optimize_f(3, 2, 0.1, 1)[0] - 3) < 1, "Hint: you are probably taking steps too large"
```

Tune the parameters

We will now try if our optimization method works.

Use this helper function to plot the results:

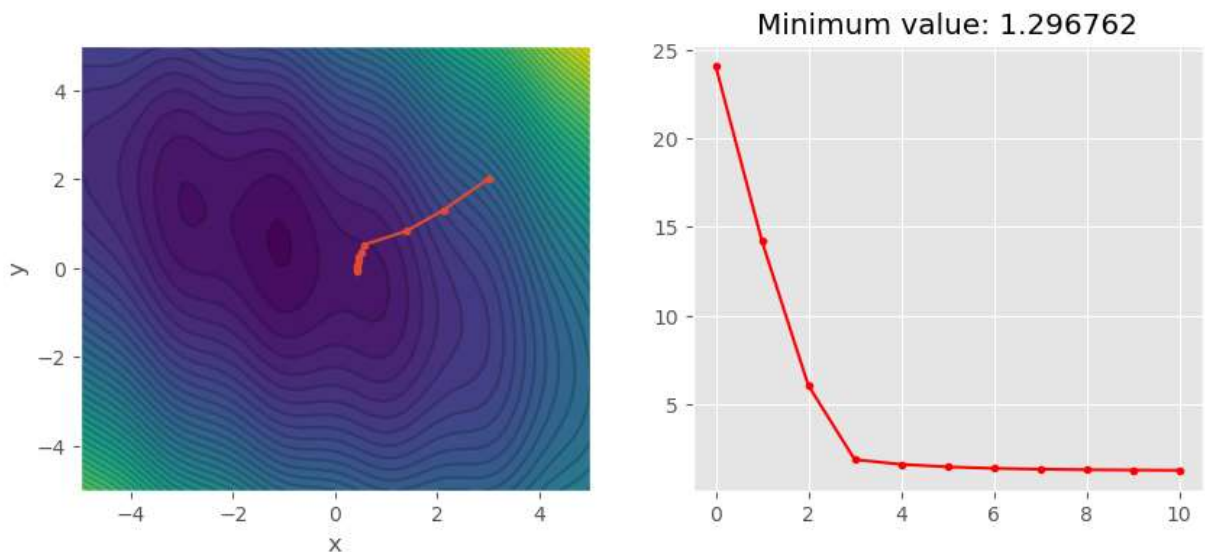
```
In [ ]: # helper function that plots the results of the gradient descent optimization
        def plot_gradient_descent_results(x, y, val, x_hist, y_hist):
            # plot the path on the contour plot
            plt.figure(figsize=(10, 4))
            plt.subplot(1, 2, 1)
            plot_f_contours()
            plt.plot(x_hist, y_hist, '-.')

            # plot the learning curve
            plt.subplot(1, 2, 2)
            plt.plot(f(np.array(x_hist), np.array(y_hist)), '-.r')
            plt.title('Minimum value: %f % f(x_hist[-1], y_hist[-1])')
```

(b) Run the gradient descent optimization with the following initial settings:

```
x=3, y=2, step_size=0.1, steps=10
```

```
In [ ]: results = optimize_f(x=3, y=2, step_size=0.1, steps=10)
        plot_gradient_descent_results(*results)
```

(c) Does it find the minimum of the function? What happens? (1 point)

It does not find the minimum of the function; it gets stuck in a local minimum. Changing the parameters, e.g. `step_size = 0.15`, gives the correct result, as I suppose we will see in the continuation.

(d) Try a few different values for the `step_size` and the number of `steps` to get close to the optimal solution:

```
In [ ]: # set parameters to search through
step_sizes = [0.02, 0.05, 0.1, 0.15, 0.2, 0.3]
n_steps = [5, 10, 20, 30, 40]
all_results = []

# set up meshgrid to ease searching
meshstuff = np.meshgrid(step_sizes, n_steps)
meshstuff = np.array(meshstuff).reshape(2, len(step_sizes)*len(n_steps)).T

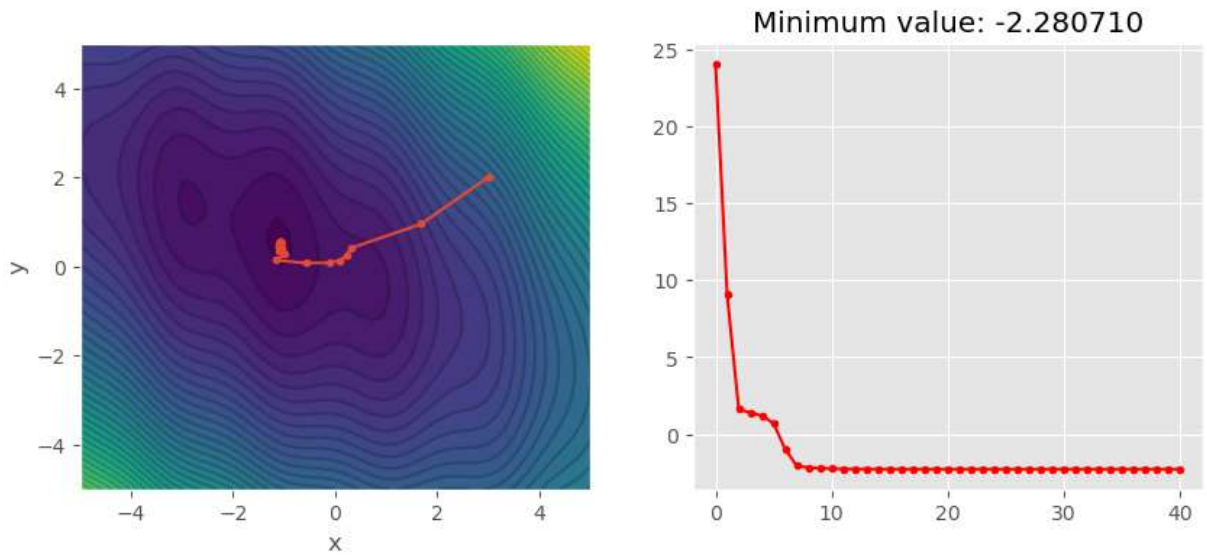
# calculate results for each combination of parameters
for i, args in enumerate(meshstuff):
    step_size, steps = args
    results = optimize_f(x=3, y=2, step_size=step_size, steps=int(steps))
    all_results.append([step_size, steps, results[2]])

# find the best solution and accompanying parameters
all_results = np.array(all_results)
minval = np.argmin(all_results[:,2])
optimal_parameters = all_results[minval]

# plot best solution
results = optimize_f(x=3, y=2, step_size=optimal_parameters[0], steps=int(optimal_p
plot_gradient_descent_results(*results))

print(f"Best solution: f(x,y) = {optimal_parameters[2]} using parameters: step_size
```

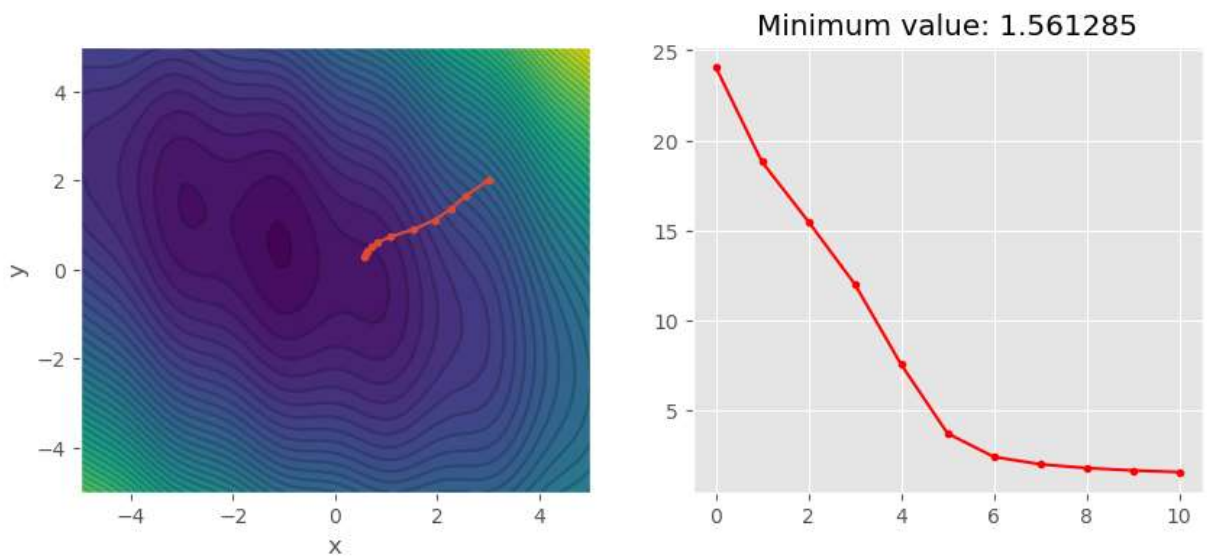
Best solution: $f(x,y) = -2.280709708105346$ using parameters: $\text{step_size} = 0.15$, $\text{steps} = 40.0$

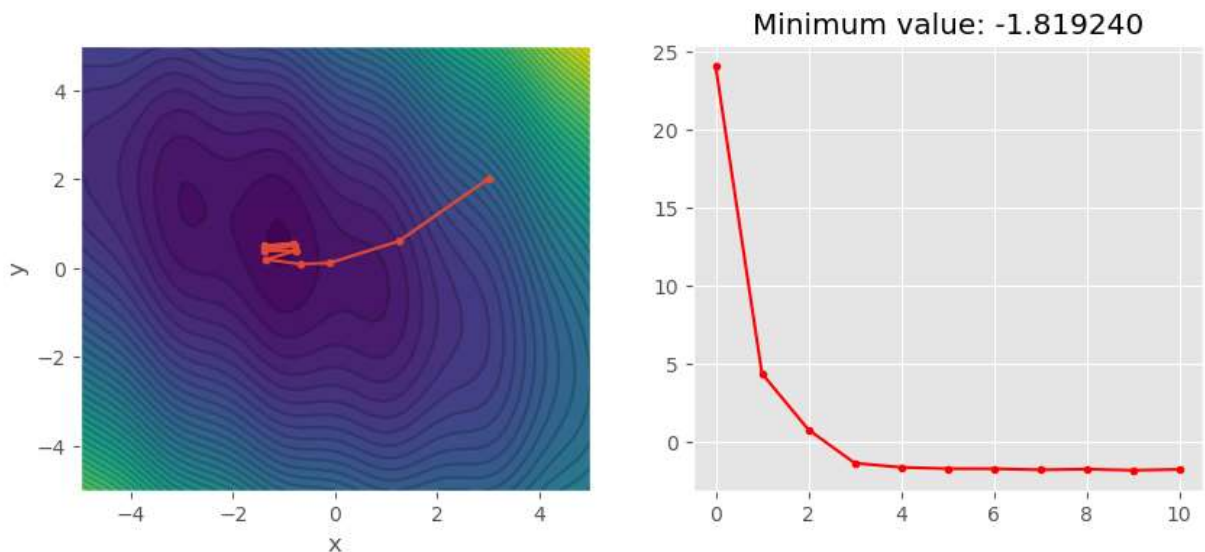


(e) What happens if you set the step size too small? And what if it is too large?(1 point)

```
In [ ]: results = optimize_f(x=3, y=2, step_size=0.05, steps=10)
         plot_gradient_descent_results(*results)

         results = optimize_f(x=3, y=2, step_size=0.2, steps=10)
         plot_gradient_descent_results(*results)
```





It is visible that setting the step size too small leads to getting stuck in local minima and a higher number of steps would be needed, whereas setting it too big makes it bounce around the optimal solution.

(f) Were you able to find a step size that reached the global optimum? If not, why not? (1 point)

Yes

Implement a decreasing step size

You might get better results if you use a step size that is large at the beginning, but slowly decreases during the optimization.

Try the following scheme to compute the step size η_t in step t , given a decay parameter d :

$$\eta_t = \eta_0 d^t \quad (4)$$

(g) Update your optimization function to use this step size schedule: (1 point)

```
In [ ]: def optimize_f(x, y, step_size, steps, decay=1.0):
    # keep track of the parameters we tried so far
    x_hist, y_hist = [x], [y]

    # run gradient descent for the number of steps
    for step in range(steps):
        # compute the gradients at this point
        dx = grad_x_f(x, y)
        dy = grad_y_f(x, y)

        # apply the gradient descent updates to x and y
        x -= step_size * dx
        y -= step_size * dy

        # store the new parameters
```

```

        x_hist.append(x)
        y_hist.append(y)

        # decay the step size
        step_size *= decay

    return x, y, f(x, y), x_hist, y_hist

```

```

In [ ]: # The following assert statement checks that your implementation behaves sensibly
_trace = optimize_f(0.123, 0.456, 0.01, 2, 0.1)[3]
assert abs(_trace[1] - _trace[0]) > 5 * abs(_trace[2] - _trace[1]), "Hint: step siz
del _trace

```

(h) Tune the `step_sizes`, `steps` and `decay` parameters to get closer to the global minimum: (1 point)

```

In [ ]: # set parameters to search through
step_sizes = [0.02, 0.05, 0.1, 0.15, 0.2, 0.3]
n_steps = [5, 10, 20, 30, 40]
decay = [0.5, 0.75, 0.9, 0.95, 0.99, 1.0]
all_results = []

# set up meshgrid to ease searching
meshstuff = np.meshgrid(step_sizes, n_steps, decay)
meshstuff = np.array(meshstuff).reshape(3, len(decay)*len(step_sizes)*len(n_steps))

# calculate results for each combination of parameters
for i, args in enumerate(meshstuff):
    step_size, steps, decay_val = args
    results = optimize_f(x=3, y=2, step_size=step_size, steps=int(steps), decay = d
    all_results.append([step_size, steps, decay_val, results[2]])

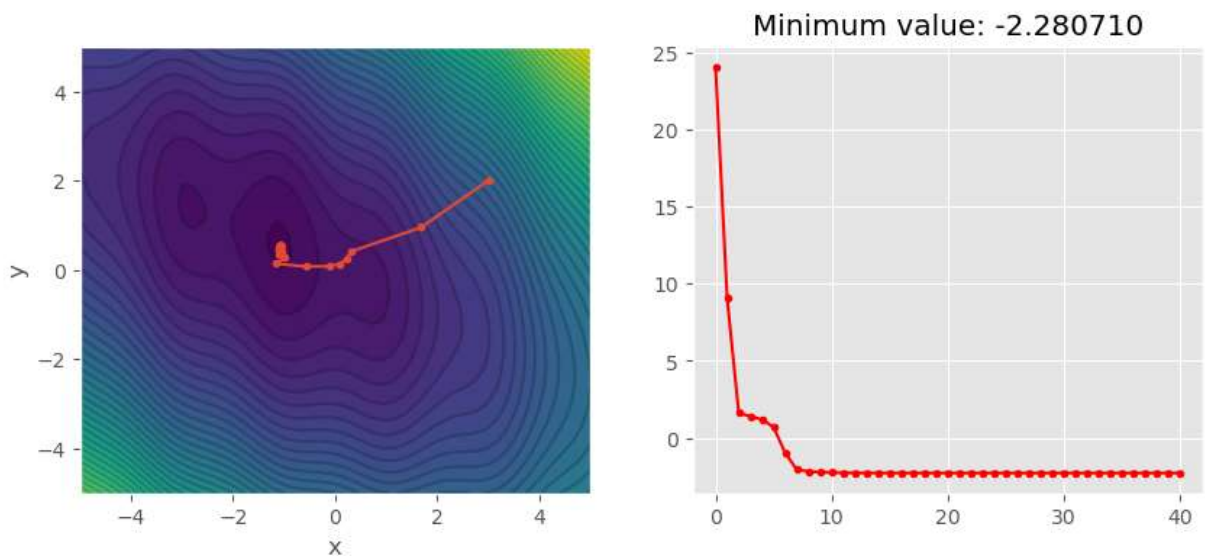
# find the best solution and accompanying parameters
all_results = np.array(all_results)
minval = np.argmin(all_results[:,3])
optimal_parameters = all_results[minval]

# plot best solution
results = optimize_f(x=3, y=2, step_size=optimal_parameters[0], steps=int(optimal_p
plot_gradient_descent_results(*results)

print(f"Best solution: f(x,y) = {optimal_parameters[3]} using parameters: decay = {

```

Best solution: $f(x,y) = -2.280709708105346$ using parameters: decay = 1.0, step_size = 0.15, steps = 40.0



The decay option does not seem to improve the solution, given that the optimal parameter set has decay = 1.0

It does not really improve it as the one that was obtained before was too exact, but if we wouldn't have checked that much this might have been better.

1.2 Neural network components (16 points)

In this assignment, we will implement a simple neural network from scratch. We need four components:

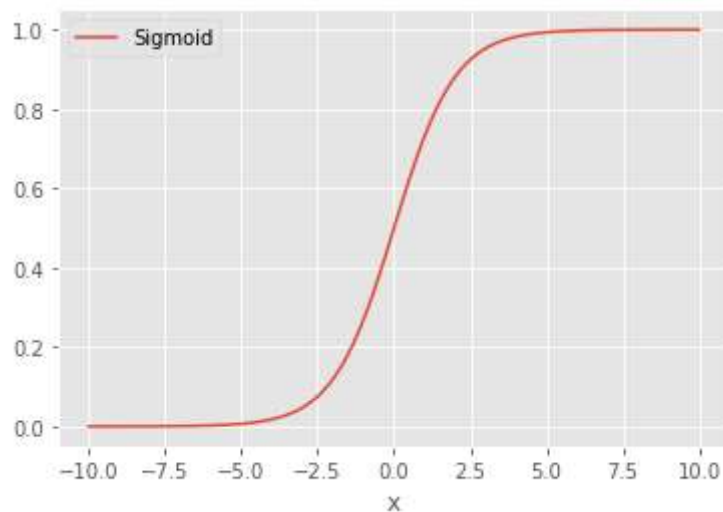
1. A sigmoid activation function,
2. A ReLU activation function,
3. A binary cross-entropy loss function,
4. A linear layer.

For each component, we will implement the forward pass, the backward pass, and the gradient descent update.

Sigmoid non-linearity

The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (5)$$



(a) Give the derivative of the sigmoid function:

(1 point)

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x) * (1 - \sigma(x)) \quad (6)$$

(b) Implement the sigmoid and its gradient in the functions `sigmoid(x)` and `sigmoid_grad(x)` :

(2 points)

```
In [ ]: def sigmoid(x):
        return 1 / (1 + np.exp(-x))

        def sigmoid_grad(x):
            return sigmoid(x) * (1 - sigmoid(x))

        # try with a random input
        rng = np.random.default_rng(12345)
        x = rng.uniform(-10, 10, size=5)
        print('x:', x)
        print('sigmoid(x):', sigmoid(x))
        print('sigmoid_grad(x):', sigmoid_grad(x))
```

```
x: [-5.45328 -3.664833  5.947309  3.525093 -2.177809]
sigmoid(x): [0.004264 0.024969 0.997394 0.971393 0.101761]
sigmoid_grad(x): [0.004246 0.024346 0.002599 0.027788 0.091406]
```

To check that the gradient implementation is correct, we can compute the numerical derivative using the [finite difference](#) method. From [Chapter 11.5 of the Deep Learning book](#):

Because

$$f'(x) = \lim_{\epsilon \rightarrow 0} \frac{f(x + \epsilon) - f(x)}{\epsilon}, \quad (7)$$

we can approximate the derivative by using a small, finite ϵ :

$$f'(x) \approx \frac{f(x + \epsilon) - f(x)}{\epsilon}. \quad (8)$$

We can improve the accuracy of the approximation by using the centered difference:

$$f'(x) \approx \frac{f(x + \frac{1}{2}\epsilon) - f(x - \frac{1}{2}\epsilon)}{\epsilon}. \quad (9)$$

The perturbation size ϵ must be large enough to ensure that the perturbation is not rounded down too much by finite-precision numerical computations.

(c) Use the central difference method to check your implementation of the sigmoid gradient. Compute the numerical gradient and check that it is close to the symbolic gradient computed by your implementation: (1 point)

```
In [ ]: # start with some random inputs
rng = np.random.default_rng(12345)
x = rng.uniform(-2, 2, size=5)

# compute the symbolic gradient
print('Symbolic ', sigmoid_grad(x))

eps = 1e-5
num_grad = (sigmoid(x+0.5*eps) - sigmoid(x-0.5*eps))/eps

print('Numerical', num_grad)
```

```
Symbolic [0.188245 0.219215 0.178901 0.221338 0.238508]
Numerical [0.188245 0.219215 0.178901 0.221338 0.238508]
```

(d) Is the gradient computed with finite differences exactly the same as the analytic answer? Why (not)? (1 point)

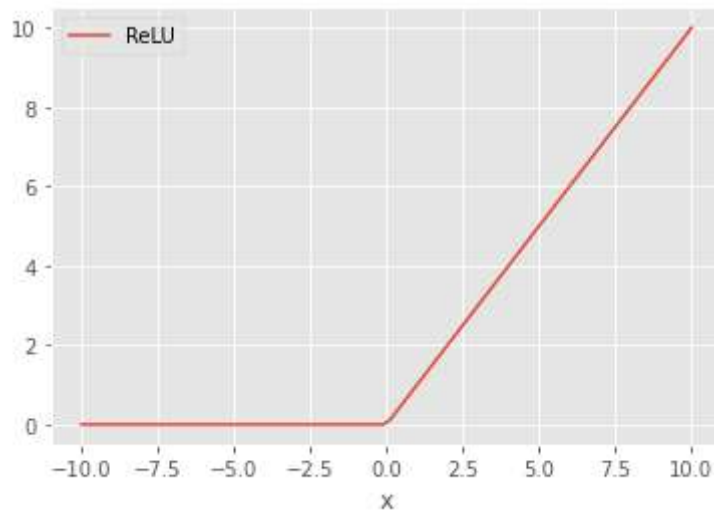
Up to 6 decimals it is exactly the same for all cases; it follows naturally that it is the same, since the same symbolic quantities are calculated, where one is exact and the other is the approximated version.

If there is a big difference between the two gradients, please try to make this as small as possible before you continue.

Rectified linear units (ReLU)

The rectified linear unit is defined as:

$$f(x) = \max(0, x) \quad (10)$$



(e) Give the derivative of the ReLU function:

(1 point)

Note: this gradient is not well-defined everywhere, but make a sensible choice for all values of x .

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$

(f) Implement the ReLU function and its gradient in the functions `relu(x)` and `relu_grad(x)`. Use the finite difference method to check that the gradient is correct:

(2 points)

```
In [ ]: def relu(x):
        return np.maximum(0, x)

def relu_grad(x):
    return (x > 0).astype(float)

# try with a random input
rng = np.random.default_rng(12345)
x = rng.uniform(-10, 10, size=5)
print('x:', x)
print('relu(x):', relu(x))
print('relu_grad(x):', relu_grad(x))
print()

# compute and compare the symbolic and numerical gradients
eps = 1e-5
num_grad = (relu(x+0.5*eps) - relu(x-0.5*eps))/eps
print('Symbolic ', relu_grad(x))
print('Numerical', num_grad)
```

```
x: [-5.45328 -3.664833  5.947309  3.525093 -2.177809]
relu(x): [0.          0.          5.947309  3.525093  0.          ]
relu_grad(x): [0.  0.  1.  1.  0.]
```

```
Symbolic [0.  0.  1.  1.  0.]
Numerical [0.  0.  1.  1.  0.]
```

Comparing sigmoid and ReLU

The sigmoid and ReLU activation functions have slightly different characteristics.

(g) Run the code below to plot the sigmoid and ReLU activation functions and their gradients:

```
In [ ]: x = np.linspace(-10, 10, 100)

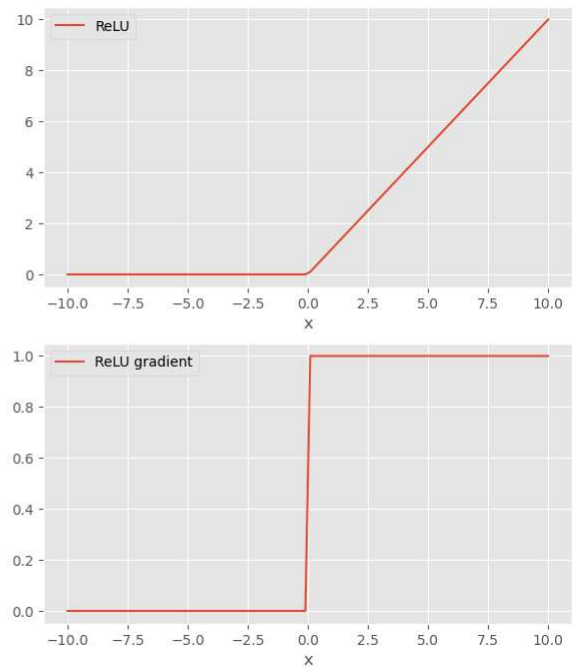
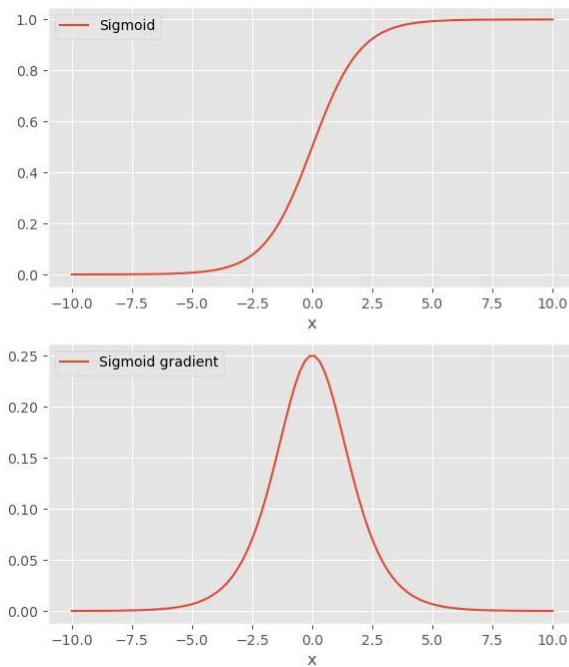
plt.figure(figsize=(15, 8))

plt.subplot(2, 2, 1)
plt.plot(x, sigmoid(x), label='Sigmoid')
plt.xlabel('x')
plt.legend(loc='upper left')

plt.subplot(2, 2, 2)
plt.plot(x, relu(x), label='ReLU')
plt.xlabel('x')
plt.legend(loc='upper left')

plt.subplot(2, 2, 3)
plt.plot(x, sigmoid_grad(x), label='Sigmoid gradient')
plt.xlabel('x')
plt.legend(loc='upper left')

plt.subplot(2, 2, 4)
plt.plot(x, relu_grad(x), label='ReLU gradient')
plt.xlabel('x')
plt.legend(loc='upper left');
```



(h) Which activation function would you recommend for a network that outputs probabilities, i.e., outputs $\in [0, 1]$? Why? (1 point)

For the use case outputs $\in [0, 1]$ using the sigmoid activation function is preferred since the sigmoid itself is bounded by those values as well

(i) Compare the gradients for sigmoid and ReLU. What are the advantages and disadvantages of each activation function in terms of their gradient? (1 point)

Sigmoid advantages:

- smooth function, making it exactly differentiable;
- bounded between 0 and 1

Sigmoid disadvantages:

- Vanishing gradient for very small values;
- Computationally expensive

ReLU advantages:

- Simple function, thus computationally efficient

ReLU disadvantages:

- sets all negative values to 0
- Non-differentiable at 0

Binary cross-entropy loss

We will use the binary cross-entropy loss to train our network. This loss function is useful for binary classification.

The binary cross-entropy (BCE) is a function of the ground truth label $y \in \{0, 1\}$ and the predicted label $\hat{y} \in [0, 1]$:

$$\mathcal{L} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})) \quad (11)$$

To minimize the BCE loss with gradient descent, we need to compute the gradient with respect to the prediction $\hat{y}$.

(j) Derive the gradient for the BCE loss: **(1 point)**

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \quad (12)$$

(k) Implement `bce_loss(y, y_hat)` and `bce_loss_grad(y, y_hat)` and use the finite difference method to check that the gradient is correct: **(3 points)**

```
In [ ]: def bce_loss(y, y_hat):
        return -(y * np.log(y_hat) + (1 - y) * np.log(1 - y_hat))

def bce_loss_grad(y, y_hat):
    return -(y/y_hat) + (1 - y)/(1 - y_hat)

# try with some random inputs
rng = np.random.default_rng(12345)
y = rng.integers(2, size=5)
y_hat = rng.uniform(0, 1, size=5)
print('y:', y)
print('y_hat:', y_hat)
print('bceloss(y, y_hat):', bce_loss(y, y_hat))
print()

# compute and compare the symbolic and numerical gradients
eps = 1e-5
num_grad = (bce_loss(y, y_hat+0.5*eps) - bce_loss(y, y_hat-0.5*eps))/eps
print('Symbolic ', bce_loss_grad(y, y_hat))
print('Numerical', num_grad)
```

```
y: [1 0 1 0 0]
y_hat: [0.676255 0.39111 0.332814 0.598309 0.186734]
bceloss(y, y_hat): [0.391186 0.496117 1.100172 0.912072 0.206697]
```

```
Symbolic [-1.478733 1.642332 -3.004682 2.489474 1.22961 ]
Numerical [-1.478733 1.642332 -3.004682 2.489474 1.22961 ]
```

Linear layer

Finally, we need to compute the gradients for the linear layer in our network.

Define a linear model $\mathbf{y} = \mathbf{x}\mathbf{W} + \mathbf{b}$, where

- $\mathbf{x}$ is an input vector of shape N ,
- $\mathbf{W}$ is a weight matrix of shape $N \times M$,
- $\mathbf{b}$ is a bias vector of shape M ,
- $\mathbf{y}$ is the output vector of shape M .

(l) Derive the gradients for $\mathbf{y}$ with respect to the input $\mathbf{x}$ and the parameters $\mathbf{W}$ and $\mathbf{b}$:
(1 point)

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{W} \quad \frac{\partial \mathbf{y}}{\partial \mathbf{W}} = \mathbf{x} \quad \frac{\partial \mathbf{y}}{\partial \mathbf{b}} = \mathbf{1}_M$$

(m) Given the gradient $\nabla_{\mathbf{y}}\mathcal{L}$ for the loss w.r.t. $\mathbf{y}$, use the chain rule to derive the gradients for the loss w.r.t. $\mathbf{x}$, $\mathbf{W}$ and $\mathbf{b}$:
(1 point)

$$\begin{aligned}\nabla_{\mathbf{x}}\mathcal{L} &= \nabla_{\mathbf{y}}\mathcal{L} \cdot \mathbf{W} \\ \nabla_{\mathbf{W}}\mathcal{L} &= \mathbf{x}^T \cdot \nabla_{\mathbf{y}}\mathcal{L} \\ \nabla_{\mathbf{b}}\mathcal{L} &= \nabla_{\mathbf{y}}\mathcal{L}\end{aligned}$$

1.3 Implement a one-layer model (2 points)

We can now implement a simple one-layer model with a sigmoid activation:

1. Given an input vector $\mathbf{x}$, weight vector $\mathbf{w}$ and bias b , compute the output $\hat{y}$:

$$h = \mathbf{x}\mathbf{w}^T + b \tag{13}$$

$$\hat{y} = \sigma(h) \tag{14}$$

2. Compute the BCE loss comparing the prediction $\hat{y}$ with the ground-truth label y .
3. Compute the gradient for the BCE loss and back-propagate this to get $\nabla_{\mathbf{x}}\mathcal{L}$, the gradient of $\mathcal{L}$ w.r.t. $\mathbf{x}$.

Hint: in numpy inner product and matrix multiplication is denoted as `np.dot(A, B)` or as `A @ B`.

(a) Complete the implementation below: (2 points)

```
In [ ]: # initialize parameters
rng = np.random.default_rng(12345)
w = rng.normal(size=5)
b = rng.normal()

# implement the model
def fn(x, y):
    h = x @ w.T + b
```



```

y_hat = sigmoid(h)
loss = bce_loss(y, y_hat)

# compute the gradients
grad_y_hat = bce_loss_grad(y, y_hat)
grad_h = sigmoid_grad(h) * grad_y_hat
grad_x = grad_h * w

return loss, grad_x

# test with a random input
x = rng.uniform(size=5)
y = 1

loss, grad_x = fn(x, y)
print("Loss", loss)
print("Gradient", grad_x)

assert np.isscalar(loss), "Loss should be scalar"
assert grad_x.shape == x.shape, "Gradient should have same shape as x"

```

Loss 2.309880244091049

Gradient [1.282477 -1.138274 0.784228 0.233444 0.067864]

(b) Use the finite-difference method to check the gradient $\nabla_x \mathcal{L}$:

```

In [ ]: # start with some random inputs
rng = np.random.default_rng(12345)
x = rng.uniform(size=5)
y = 1

# set epsilon to a small value
eps = 0.00001

numerical_grad = np.zeros(x.shape)
# compute the gradient for each element of x separately
for i in range(len(x)):
    # compute inputs at -eps/2 and +eps/2
    x_a, x_b = x.copy(), x.copy()
    x_a[i] += eps / 2
    x_b[i] -= eps / 2

    # compute the gradient for this element
    loss_a, _ = fn(x_a, y)
    loss_b, _ = fn(x_b, y)
    numerical_grad[i] = (loss_a - loss_b) / eps

# compute the symbolic gradient
loss, symbolic_grad = fn(x, y)

print("Symbolic gradient")
print(symbolic_grad)
print("Numerical gradient")
print(numerical_grad)

```

Symbolic gradient

```
[ 1.177245 -1.044874  0.719879  0.214289  0.062295]
```

Numerical gradient

```
[ 1.177245 -1.044874  0.719879  0.214289  0.062295]
```

1.4 Implement a linear layer and the sigmoid and ReLU activation functions (5 points)

We will now construct a simple neural network. We need to implement the following objects:

- **Linear** : a layer that computes $y = x \cdot W + b$.
- **Sigmoid** : a layer that computes $y = \text{sigmoid}(x)$.
- **ReLU** : a layer that computes $y = \text{relu}(x)$.

For each layer class, we need to implement the following methods:

- **forward** : The forward pass that computes the output y given x .
- **backward** : The backward pass that receives the gradient for y and computes the gradients for the input x and the parameters of the layer.
- **step** : The update step that applies the gradient updates to the parameters of the layer, based on the gradient computed and stored by **backward**.

(a) Implement a class `Linear` that computes $y = x \cdot W + b$:

(3 points)

$$\begin{aligned}\nabla_x \mathcal{L} &= \nabla_y \mathcal{L} \cdot W \\ \nabla_W \mathcal{L} &= x^T \cdot \nabla_y \mathcal{L} \\ \nabla_b \mathcal{L} &= \nabla_y \mathcal{L}\end{aligned}$$

```
In [ ]: class Linear:
    def __init__(self, n_in, n_out, rng = np.random.default_rng(12345)):
        # initialize the weights randomly,
        # using the Xavier initialization rule for scale
        a = np.sqrt(6 / (n_in * n_out))
        self.W = rng.uniform(-a, a, size=(n_in, n_out))
        self.b = np.zeros((n_out,))

    def forward(self, x):
        y = x @ self.W + self.b
        return y

    def backward(self, x, dy):
        # TODO: compute the backward pass,
        # given dy, compute the gradients for x, W and b

        dx = dy @ self.W.T
        self.dW = x.T @ dy
        self.db = np.sum(dy, axis=0)
        return dx
```

```

def step(self, step_size):
    # TODO: apply a gradient descent update step
    self.W -= step_size * self.dW
    self.b -= step_size * self.db

def __str__(self):
    return 'Linear %dx%d' % self.W.shape

# Try the new class with some random values.
# Debugging tip: always choose a unique length for each dimension,
# so you'll get an error if you mix them up.
rng = np.random.default_rng(12345)
x = rng.uniform(size=(3, 5))

layer = Linear(5, 7, rng=rng)
y = layer.forward(x)
dx = layer.backward(x, np.ones_like(y))
print('y:', y)
print('dx:', dx)

# Verify correctness
assert y.shape == (3,7)
assert dx.shape == x.shape
layer.W *= 2
layer.b = layer.b * 2 + 1
y2 = layer.forward(x)
dx2 = layer.backward(x, np.ones_like(y))
assert np.all(y2 == 2 * y + 1)
assert np.all(dx2 == 2 * dx)

```

```

y: [[ 0.252427  0.428688 -0.081518  0.207314 -0.058535 -0.005779  0.059303]
 [ 0.382911  0.146397 -0.275544 -0.026378 -0.333927 -0.537221 -0.223564]
 [ 0.15955   0.155119 -0.222059  0.428698 -0.231045 -0.345936 -0.119919]]
dx: [[-0.326296 -0.992105  1.657474  0.165888 -0.622481]
 [-0.326296 -0.992105  1.657474  0.165888 -0.622481]
 [-0.326296 -0.992105  1.657474  0.165888 -0.622481]]

```

(b) Implement a class `Sigmoid` that computes $y = 1 / (1 + \exp(-x))$: (1 point)

```

In [ ]: class Sigmoid:
    def forward(self, x):
        return sigmoid(x)

    def backward(self, x, dy):
        dx = dy * sigmoid_grad(x)
        return dx

    def step(self, step_size):
        pass

    def __str__(self):
        return 'Sigmoid'

```

```

# try the new class with some random values
rng = np.random.default_rng(12345)
x = rng.normal(size=(3, 5))

layer = Sigmoid()
y = layer.forward(x)
dx = layer.backward(x, np.ones_like(y))
print('y:', y)
print('dx:', dx)

assert y.shape == x.shape, "Output sigmoid should have the same shape as input"
assert dx.shape == x.shape, "Gradient sigmoid should have the same shape as input"
assert np.all(y > 0) and np.all(y < 1), "Output of sigmoid should be between 0 and 1"

```

```

y: [[0.194063 0.779667 0.295117 0.435567 0.481173]
     [0.322811 0.202977 0.656761 0.589297 0.124242]
     [0.912728 0.72482  0.318779 0.711401 0.385338]]
dx: [[0.156402 0.171786 0.208023 0.245848 0.249646]
      [0.218604 0.161777 0.225426 0.242026 0.108806]
      [0.079656 0.199456 0.217159 0.20531  0.236853]]

```

(c) Implement a class `ReLU` that computes $y = \max(0, x)$:

(1 point)

```

In [ ]: # Computes y = max(0, x).
class ReLU:
    def forward(self, x):
        return relu(x)

    def backward(self, x, dy):
        return dy * relu_grad(x)

    def step(self, step_size):
        pass

    def __str__(self):
        return 'ReLU'

```

```

# try the new class with some random values
rng = np.random.default_rng(12345)
x = rng.uniform(-10, 10, size=(3, 5))

layer = ReLU()
y = layer.forward(x)
dx = layer.backward(x, np.ones_like(y))
print('y:', y)
print('dx:', dx)

assert y.shape == x.shape, "Output of ReLU should have the same shape as input"
assert dx.shape == x.shape, "Gradient of ReLU should have the same shape as input"

```

```

y: [[0.          0.          5.947309  3.525093  0.          ]
     [0.          1.966175  0.          3.455121  8.836057]
     [0.          8.977623  3.344749  0.          0.          ]]
dx: [[0.  0.  1.  1.  0.]
      [0.  1.  0.  1.  1.]
      [0.  1.  1.  0.  0.]]

```

Verify the gradients

The code below will check your implementations using SciPy's finite difference implementation `check_grad`. This is similar to what we did manually before, but automates some of the work.

(d) Run the code and check that the error is not too large.

```

In [ ]: ## Verify gradient computations for Linear
# test for dx
rng = np.random.default_rng(12345)
layer = Linear(5, 7, rng)
def test_fn(x):
    x = x.reshape(3, 5)
    # multiply the output with a constant to check if
    # the gradient uses dy
    return 2 * np.sum(layer.forward(x))
def test_fn_grad(x):
    x = x.reshape(3, 5)
    # multiply the incoming dy gradient with a constant
    return layer.backward(x, 2 * np.ones((3, 7))).flatten()

err = scipy.optimize.check_grad(test_fn, test_fn_grad, rng.uniform(-10, 10, size=3))
print("err on dx:", err)
assert np.abs(err) < 1e-5, "Error on dx is too large, check your implementation of"

# test for dW
x = rng.uniform(size=(3, 5))
layer = Linear(5, 7, rng)
def test_fn(w):
    layer.W = w.reshape(5, 7)
    # multiply the output with a constant to check if
    # the gradient uses dy
    return 2 * np.sum(layer.forward(x))
def test_fn_grad(w):
    layer.W = w.reshape(5, 7)
    # multiply the incoming dy gradient with a constant
    layer.backward(x, 2 * np.ones((3, 7)))
    return layer.dW.flatten()

err = scipy.optimize.check_grad(test_fn, test_fn_grad, rng.uniform(-10, 10, size=5))
print("err on dW:", err)
assert np.abs(err) < 1e-5, "Error on dW is too large, check your implementation of"

# test for db
x = rng.uniform(size=(3, 5))
layer = Linear(5, 7, rng)

```

```

def test_fn(b):
    layer.b = b
    # multiply the output with a constant to check if
    # the gradient uses dy
    return 2 * np.sum(layer.forward(x))
def test_fn_grad(b):
    layer.b = b
    # multiply the incoming dy gradient with a constant
    layer.backward(x, 2 * np.ones((x.shape[0], 7)))
    return layer.db

err = scipy.optimize.check_grad(test_fn, test_fn_grad, rng.uniform(-10, 10, size=7))
print("err on db:", err)
assert np.abs(err) < 1e-5, "Error on db is too large, check your implementation of

```

err on dx: 1.1760260802457315e-06
 err on dW: 1.4712410588276974e-06
 err on db: 0.0

```

In [ ]: ## Verify gradient computation for Sigmoid
        # test for dx
        layer = Sigmoid()
        def test_fn(x):
            # multiply the output with a constant to check if
            # the gradient uses dy
            return np.sum(2 * layer.forward(x))
        def test_fn_grad(x):
            # multiply the incoming dy gradient with a constant
            return layer.backward(x, 2 * np.ones(x.shape))

        rng = np.random.default_rng(12345)
        err = scipy.optimize.check_grad(test_fn, test_fn_grad, rng.uniform(-10, 10, size=5))
        print("err on dx:", err)
        assert np.abs(err) < 1e-5, "Error on dx is too large, check your implementation of

```

err on dx: 4.8238536342998094e-08

```

In [ ]: ## Verify gradient computation for ReLU
        # test for dx
        layer = ReLU()
        def test_fn(x):
            # multiply the output with a constant to check if
            # the gradient uses dy
            return 2 * np.sum(layer.forward(x))
        def test_fn_grad(x):
            # multiply the incoming dy gradient with a constant
            return layer.backward(x, 2 * np.ones(x.shape))

        rng = np.random.default_rng(12345)
        err = scipy.optimize.check_grad(test_fn, test_fn_grad, rng.uniform(1, 10, size=5))
        print("err on dx:", err)
        assert np.abs(err) < 1e-5, "Error on dx is too large, check your implementation of

```

err on dx: 0.0

1.5 Construct a neural network with back-propagation

We will use the following container class to implement the network:

1. The `forward` pass computes the output of each layer. We store the intermediate inputs for the backward pass.
2. The `backward` pass computes the gradients for each layer, in reverse order, by using the original input `x` and the gradient `dy` from the previous layer.
3. The `step` function will ask each layer to apply the gradient descent updates to its weights.

(a) Read the code below:

```
In [ ]: class Net:
    def __init__(self, layers):
        self.layers = layers

    def forward(self, x):
        # compute the forward pass for each Layer
        trace = []
        for layer in self.layers:
            # compute the forward pass
            y = layer.forward(x)
            # store the original input for the backward pass
            trace.append((layer, x))
            x = y
        # return the final output and the history trace
        return y, trace

    def backward(self, trace, dy):
        # compute the backward pass for each Layer
        for layer, x in trace[::-1]:
            # compute the backward pass using the original input x
            dy = layer.backward(x, dy)

    def step(self, learning_rate):
        # apply the gradient descent updates of each Layer
        for layer in self.layers:
            layer.step(learning_rate)

    def __str__(self):
        return '\n'.join(str(l) for l in self.layers)
```

1.6 Training the network (10 points)

We load a simple dataset with 360 handwritten digits.

Each sample has 8×8 pixels, arranged as a 1D vector of 64 features.

We create a binary classification problem with the label 0 for the digits 0 to 4, and 1 for the digits 5 to 9.

```
In [ ]: # Load the first two classes of the digits dataset
dataset = sklearn.datasets.load_digits()
digits_x, digits_y = dataset['data'], dataset['target']

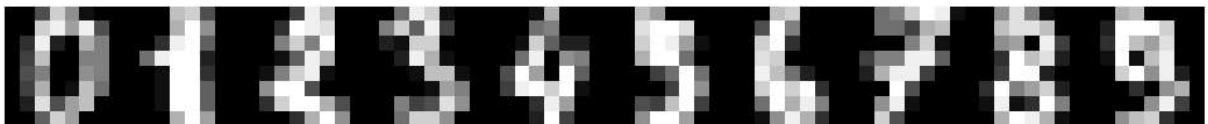
# create a binary classification problem
digits_y = (digits_y < 5).astype(float)

# plot some of the digits
plt.figure(figsize=(10, 2))
plt.imshow(np.hstack([digits_x[i].reshape(8, 8) for i in range(10)]), cmap='gray')
plt.grid(False)
plt.tight_layout()
plt.axis('off')

# normalize the values to [0, 1]
digits_x -= np.mean(digits_x)
digits_x /= np.std(digits_x)

# print some statistics
print('digits_x.shape:', digits_x.shape)
print('digits_y.shape:', digits_y.shape)
print('min, max values:', np.min(digits_x), np.max(digits_x))
print('labels:', np.unique(digits_y))
```

```
digits_x.shape: (1797, 64)
digits_y.shape: (1797,)
min, max values: -0.8117561971974786 1.847470154168513
labels: [0. 1.]
```



We divide the dataset in a train and a test set.

```
In [ ]: # make a 50%/50% train/test split
train_prop = 0.5
n_train = int(digits_x.shape[0] * train_prop)

# shuffle the images
rng = np.random.default_rng(12345)
idxs = rng.permutation(digits_x.shape[0])

# take a subset
x = {'train': digits_x[idxs[:n_train]],
     'test': digits_x[idxs[n_train:]]}
y = {'train': digits_y[idxs[:n_train]],
     'test': digits_y[idxs[n_train:]]}

print('Training samples:', x['train'].shape[0])
print('Test samples:', x['test'].shape[0])
```

Training samples: 898

Test samples: 899

We will now implement a function that trains the network. For each epoch, it loops over all minibatches in the training set and updates the network weights. It will then compute the loss and accuracy for the test samples. Finally, it will plot the learning curves.

(a) Read through the code below.

```
In [ ]: def fit(net, x, y, epochs=25, learning_rate=0.001, mb_size=10):
        # initialize the loss and accuracy history
        loss_hist = {'train': [], 'test': []}
        accuracy_hist = {'train': [], 'test': []}

        for epoch in range(epochs):
            # initialize the loss and accuracy for this epoch
            loss = {'train': 0.0, 'test': 0.0}
            accuracy = {'train': 0.0, 'test': 0.0}

            # first train on training data, then evaluate on the test data
            for phase in ('train', 'test'):
                # compute the number of minibatches
                steps = x[phase].shape[0] // mb_size

                # Loop over all minibatches
                for step in range(steps):
                    # get the samples for the current minibatch
                    x_mb = x[phase][(step * mb_size):((step + 1) * mb_size)]
                    y_mb = y[phase][(step * mb_size):((step + 1) * mb_size), None]

                    # compute the forward pass through the network
                    pred_y, trace = net.forward(x_mb)

                    # compute the current loss and accuracy
                    loss[phase] += np.mean(bce_loss(y_mb, pred_y))
                    accuracy[phase] += np.mean((y_mb > 0.5) == (pred_y > 0.5))

                    # only update the network in the training phase
                    if phase == 'train':
                        # compute the gradient for the loss
                        dy = bce_loss_grad(y_mb, pred_y)

                        # backpropagate the gradient through the network
                        net.backward(trace, dy)

                        # update the weights
                        net.step(learning_rate)

                # compute the mean loss and accuracy over all minibatches
                loss[phase] = loss[phase] / steps
                accuracy[phase] = accuracy[phase] / steps

            # add statistics to history
            loss_hist[phase].append(loss[phase])
            accuracy_hist[phase].append(accuracy[phase])
```

```

        print('Epoch %3d: loss[train]=%.4f accuracy[train]=%.4f loss[test]=%.4f'
              (epoch, loss['train'], accuracy['train'], loss['test'], accuracy['tes

# plot the learning curves
plt.figure(figsize=(20, 5))

plt.subplot(1, 2, 1)
for phase in loss_hist:
    plt.plot(loss_hist[phase], label=phase)
plt.title('BCE loss')
plt.xlabel('Epoch')
plt.legend()

plt.subplot(1, 2, 2)
for phase in accuracy_hist:
    plt.plot(accuracy_hist[phase], label=phase)
plt.title('Accuracy')
plt.xlabel('Epoch')
plt.legend()

```

We will define a two-layer network:

- A linear layer that maps the 64 features of the input to 32 features.
- A ReLU activation function.
- A linear layer that maps the 32 features to the 1 output features.
- A sigmoid activation function that maps the output to $[0, 1]$.

(b) Train the network and inspect the results. Tune the hyperparameters to get a good result. (1 point)

```

In [ ]: # construct network
rng = np.random.default_rng(12345)
net = Net([
    Linear(64, 32, rng=rng),
    ReLU(),
    Linear(32, 1, rng=rng),
    Sigmoid())])

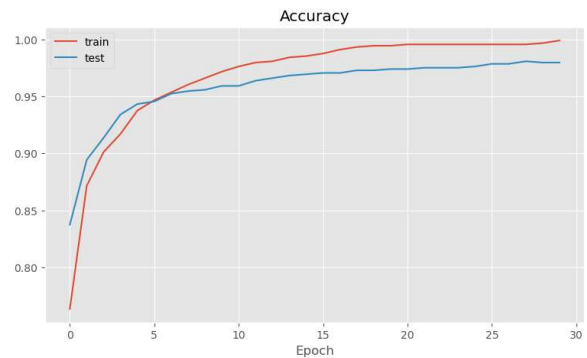
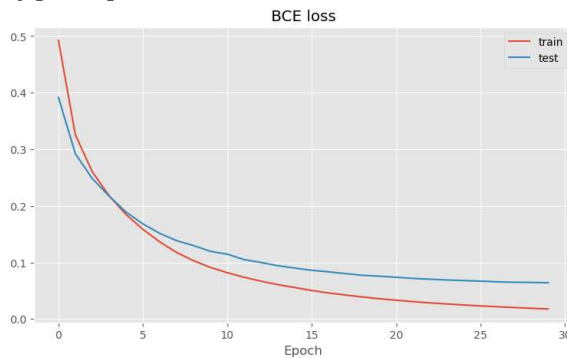
# TODO: tune the hyperparameters: DONE
fit(net, x, y,
    epochs = 30,
    learning_rate = 0.005,
    mb_size = 40)

# Note: add more cells below if you want to keep runs with different hyperparameter

```

Epoch 0:	loss[train]= 0.4922	accuracy[train]= 0.7636	loss[test]= 0.3916	accuracy[test]= 0.8375
Epoch 1:	loss[train]= 0.3256	accuracy[train]= 0.8716	loss[test]= 0.2921	accuracy[test]= 0.8943
Epoch 2:	loss[train]= 0.2603	accuracy[train]= 0.9011	loss[test]= 0.2483	accuracy[test]= 0.9136
Epoch 3:	loss[train]= 0.2178	accuracy[train]= 0.9170	loss[test]= 0.2173	accuracy[test]= 0.9341
Epoch 4:	loss[train]= 0.1847	accuracy[train]= 0.9375	loss[test]= 0.1890	accuracy[test]= 0.9432
Epoch 5:	loss[train]= 0.1585	accuracy[train]= 0.9466	loss[test]= 0.1681	accuracy[test]= 0.9455
Epoch 6:	loss[train]= 0.1363	accuracy[train]= 0.9534	loss[test]= 0.1513	accuracy[test]= 0.9523
Epoch 7:	loss[train]= 0.1175	accuracy[train]= 0.9602	loss[test]= 0.1384	accuracy[test]= 0.9545
Epoch 8:	loss[train]= 0.1031	accuracy[train]= 0.9659	loss[test]= 0.1298	accuracy[test]= 0.9557
Epoch 9:	loss[train]= 0.0911	accuracy[train]= 0.9716	loss[test]= 0.1196	accuracy[test]= 0.9591
Epoch 10:	loss[train]= 0.0818	accuracy[train]= 0.9761	loss[test]= 0.1146	accuracy[test]= 0.9591
Epoch 11:	loss[train]= 0.0737	accuracy[train]= 0.9795	loss[test]= 0.1049	accuracy[test]= 0.9636
Epoch 12:	loss[train]= 0.0668	accuracy[train]= 0.9807	loss[test]= 0.0999	accuracy[test]= 0.9659
Epoch 13:	loss[train]= 0.0606	accuracy[train]= 0.9841	loss[test]= 0.0942	accuracy[test]= 0.9682
Epoch 14:	loss[train]= 0.0556	accuracy[train]= 0.9852	loss[test]= 0.0900	accuracy[test]= 0.9693
Epoch 15:	loss[train]= 0.0503	accuracy[train]= 0.9875	loss[test]= 0.0861	accuracy[test]= 0.9705
Epoch 16:	loss[train]= 0.0459	accuracy[train]= 0.9909	loss[test]= 0.0834	accuracy[test]= 0.9705
Epoch 17:	loss[train]= 0.0424	accuracy[train]= 0.9932	loss[test]= 0.0802	accuracy[test]= 0.9727
Epoch 18:	loss[train]= 0.0389	accuracy[train]= 0.9943	loss[test]= 0.0773	accuracy[test]= 0.9727
Epoch 19:	loss[train]= 0.0359	accuracy[train]= 0.9943	loss[test]= 0.0758	accuracy[test]= 0.9739
Epoch 20:	loss[train]= 0.0332	accuracy[train]= 0.9955	loss[test]= 0.0737	accuracy[test]= 0.9739
Epoch 21:	loss[train]= 0.0307	accuracy[train]= 0.9955	loss[test]= 0.0718	accuracy[test]= 0.9750
Epoch 22:	loss[train]= 0.0284	accuracy[train]= 0.9955	loss[test]= 0.0703	accuracy[test]= 0.9750
Epoch 23:	loss[train]= 0.0266	accuracy[train]= 0.9955	loss[test]= 0.0690	accuracy[test]= 0.9750
Epoch 24:	loss[train]= 0.0248	accuracy[train]= 0.9955	loss[test]= 0.0680	accuracy[test]= 0.9761
Epoch 25:	loss[train]= 0.0231	accuracy[train]= 0.9955	loss[test]= 0.0670	accuracy[test]= 0.9784
Epoch 26:	loss[train]= 0.0216	accuracy[train]= 0.9955	loss[test]= 0.0657	accuracy[test]= 0.9784
Epoch 27:	loss[train]= 0.0203	accuracy[train]= 0.9955	loss[test]= 0.0650	accuracy[test]= 0.9807

Epoch 28: loss[train]= 0.0190 accuracy[train]= 0.9966 loss[test]= 0.0647 accuracy[test]= 0.9795
 Epoch 29: loss[train]= 0.0178 accuracy[train]= 0.9989 loss[test]= 0.0641 accuracy[test]= 0.9795



(c) How did each of the hyperparameters (number of epochs, learning rate, minibatch size) influence your results? How important is it to set each correctly? (3 points)

TODO: Your answer here: DONE

The number of epochs influence the results as long as the network is not converging, if not it will not make changes. With higher mb, it is observed that the computation time is faster, but it doesnot really change the accuracy. The learning size has a great impact, if this is big, the accuracy will decrease, but if it is too small we would need a higher numbers of epochs for a decent accuracy, therefore the importance remains in a balance between the number of epochs and a small (but not too small) learning rate. If we learn for too long the model will start overfitting with the data.

(d) Create and train a network with one linear layer followed by a sigmoid activation: (1 point)

```
net = Net([Linear(...), Sigmoid()])
```

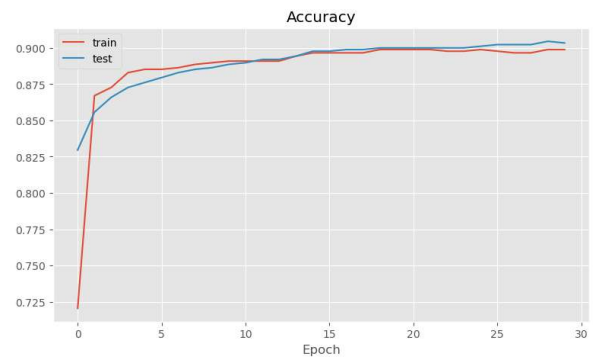
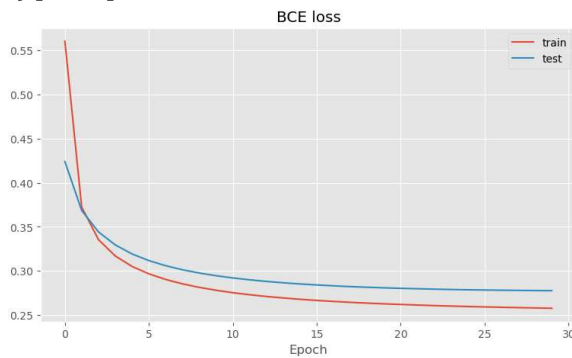
```
In [ ]: # TODO: Your code here. : DONE

rng = np.random.default_rng(12345)
net = Net([
    Linear(64, 1, rng=rng),
    Sigmoid()])

# TODO: tune the hyperparameters : DONE
fit(net, x, y,
    epochs = 30,
    learning_rate = 0.005,
    mb_size = 40)
```

Epoch 0:	loss[train]= 0.5600	accuracy[train]= 0.7205	loss[test]= 0.4238	accuracy[test]= 0.8295
Epoch 1:	loss[train]= 0.3723	accuracy[train]= 0.8670	loss[test]= 0.3685	accuracy[test]= 0.8557
Epoch 2:	loss[train]= 0.3355	accuracy[train]= 0.8727	loss[test]= 0.3442	accuracy[test]= 0.8659
Epoch 3:	loss[train]= 0.3170	accuracy[train]= 0.8830	loss[test]= 0.3295	accuracy[test]= 0.8727
Epoch 4:	loss[train]= 0.3051	accuracy[train]= 0.8852	loss[test]= 0.3193	accuracy[test]= 0.8761
Epoch 5:	loss[train]= 0.2967	accuracy[train]= 0.8852	loss[test]= 0.3117	accuracy[test]= 0.8795
Epoch 6:	loss[train]= 0.2904	accuracy[train]= 0.8864	loss[test]= 0.3059	accuracy[test]= 0.8830
Epoch 7:	loss[train]= 0.2854	accuracy[train]= 0.8886	loss[test]= 0.3014	accuracy[test]= 0.8852
Epoch 8:	loss[train]= 0.2814	accuracy[train]= 0.8898	loss[test]= 0.2977	accuracy[test]= 0.8864
Epoch 9:	loss[train]= 0.2781	accuracy[train]= 0.8909	loss[test]= 0.2946	accuracy[test]= 0.8886
Epoch 10:	loss[train]= 0.2754	accuracy[train]= 0.8909	loss[test]= 0.2921	accuracy[test]= 0.8898
Epoch 11:	loss[train]= 0.2731	accuracy[train]= 0.8909	loss[test]= 0.2900	accuracy[test]= 0.8920
Epoch 12:	loss[train]= 0.2711	accuracy[train]= 0.8909	loss[test]= 0.2882	accuracy[test]= 0.8920
Epoch 13:	loss[train]= 0.2694	accuracy[train]= 0.8943	loss[test]= 0.2866	accuracy[test]= 0.8943
Epoch 14:	loss[train]= 0.2679	accuracy[train]= 0.8966	loss[test]= 0.2853	accuracy[test]= 0.8977
Epoch 15:	loss[train]= 0.2666	accuracy[train]= 0.8966	loss[test]= 0.2841	accuracy[test]= 0.8977
Epoch 16:	loss[train]= 0.2655	accuracy[train]= 0.8966	loss[test]= 0.2832	accuracy[test]= 0.8989
Epoch 17:	loss[train]= 0.2645	accuracy[train]= 0.8966	loss[test]= 0.2823	accuracy[test]= 0.8989
Epoch 18:	loss[train]= 0.2636	accuracy[train]= 0.8989	loss[test]= 0.2816	accuracy[test]= 0.9000
Epoch 19:	loss[train]= 0.2628	accuracy[train]= 0.8989	loss[test]= 0.2809	accuracy[test]= 0.9000
Epoch 20:	loss[train]= 0.2620	accuracy[train]= 0.8989	loss[test]= 0.2803	accuracy[test]= 0.9000
Epoch 21:	loss[train]= 0.2614	accuracy[train]= 0.8989	loss[test]= 0.2798	accuracy[test]= 0.9000
Epoch 22:	loss[train]= 0.2608	accuracy[train]= 0.8977	loss[test]= 0.2794	accuracy[test]= 0.9000
Epoch 23:	loss[train]= 0.2602	accuracy[train]= 0.8977	loss[test]= 0.2790	accuracy[test]= 0.9000
Epoch 24:	loss[train]= 0.2597	accuracy[train]= 0.8989	loss[test]= 0.2787	accuracy[test]= 0.9011
Epoch 25:	loss[train]= 0.2593	accuracy[train]= 0.8977	loss[test]= 0.2784	accuracy[test]= 0.9023
Epoch 26:	loss[train]= 0.2589	accuracy[train]= 0.8966	loss[test]= 0.2782	accuracy[test]= 0.9023
Epoch 27:	loss[train]= 0.2585	accuracy[train]= 0.8966	loss[test]= 0.2780	accuracy[test]= 0.9023

Epoch 28: loss[train]= 0.2581 accuracy[train]= 0.8989 loss[test]= 0.2778 accuracy[test]= 0.9045
 Epoch 29: loss[train]= 0.2578 accuracy[train]= 0.8989 loss[test]= 0.2777 accuracy[test]= 0.9034



(e) Discuss your results. Compare the results of this single-layer network with those of the network you trained before. (1 point)

TODO: Your answer here. : DONE

In this case, using the same parameters, the results are slightly worse (around 7%). This makes sense because the network is less complex and can thus learn less complex data structures

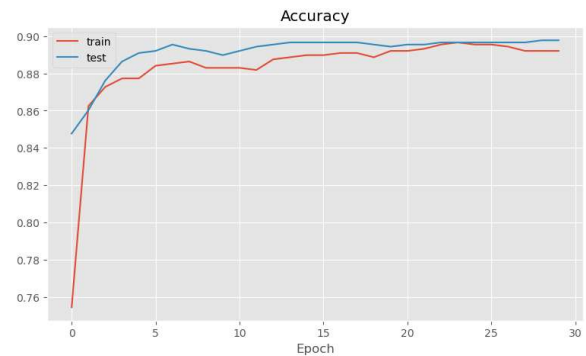
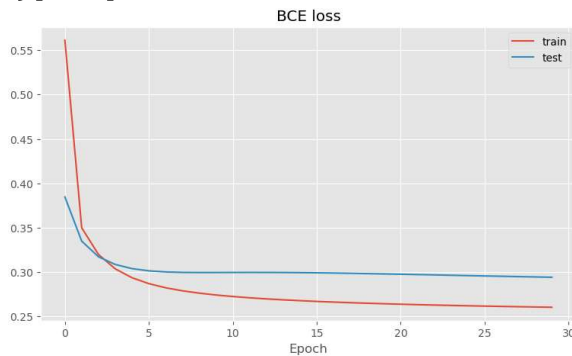
(f) Repeat the experiment with a network with two linear layers, followed by a sigmoid activation: [Linear, Linear, Sigmoid] . (1 point)

```
In [ ]: # TODO: Your code here.: DONE
# construct network
rng = np.random.default_rng(12345)
net = Net([
    Linear(64, 32, rng=rng),
    Linear(32, 1, rng=rng),
    Sigmoid()])

# TODO: tune the hyperparameters : DONE
fit(net, x, y,
    epochs = 30,
    learning_rate = 0.005,
    mb_size = 40)
```

Epoch 0:	loss[train]= 0.5611	accuracy[train]= 0.7545	loss[test]= 0.3846	accuracy[test]= 0.8477
Epoch 1:	loss[train]= 0.3499	accuracy[train]= 0.8625	loss[test]= 0.3349	accuracy[test]= 0.8602
Epoch 2:	loss[train]= 0.3199	accuracy[train]= 0.8727	loss[test]= 0.3173	accuracy[test]= 0.8761
Epoch 3:	loss[train]= 0.3036	accuracy[train]= 0.8773	loss[test]= 0.3086	accuracy[test]= 0.8864
Epoch 4:	loss[train]= 0.2936	accuracy[train]= 0.8773	loss[test]= 0.3039	accuracy[test]= 0.8909
Epoch 5:	loss[train]= 0.2870	accuracy[train]= 0.8841	loss[test]= 0.3014	accuracy[test]= 0.8920
Epoch 6:	loss[train]= 0.2824	accuracy[train]= 0.8852	loss[test]= 0.3002	accuracy[test]= 0.8955
Epoch 7:	loss[train]= 0.2789	accuracy[train]= 0.8864	loss[test]= 0.2997	accuracy[test]= 0.8932
Epoch 8:	loss[train]= 0.2763	accuracy[train]= 0.8830	loss[test]= 0.2995	accuracy[test]= 0.8920
Epoch 9:	loss[train]= 0.2742	accuracy[train]= 0.8830	loss[test]= 0.2996	accuracy[test]= 0.8898
Epoch 10:	loss[train]= 0.2725	accuracy[train]= 0.8830	loss[test]= 0.2996	accuracy[test]= 0.8920
Epoch 11:	loss[train]= 0.2711	accuracy[train]= 0.8818	loss[test]= 0.2997	accuracy[test]= 0.8943
Epoch 12:	loss[train]= 0.2698	accuracy[train]= 0.8875	loss[test]= 0.2996	accuracy[test]= 0.8955
Epoch 13:	loss[train]= 0.2688	accuracy[train]= 0.8886	loss[test]= 0.2996	accuracy[test]= 0.8966
Epoch 14:	loss[train]= 0.2678	accuracy[train]= 0.8898	loss[test]= 0.2994	accuracy[test]= 0.8966
Epoch 15:	loss[train]= 0.2670	accuracy[train]= 0.8898	loss[test]= 0.2992	accuracy[test]= 0.8966
Epoch 16:	loss[train]= 0.2662	accuracy[train]= 0.8909	loss[test]= 0.2989	accuracy[test]= 0.8966
Epoch 17:	loss[train]= 0.2656	accuracy[train]= 0.8909	loss[test]= 0.2986	accuracy[test]= 0.8966
Epoch 18:	loss[train]= 0.2649	accuracy[train]= 0.8886	loss[test]= 0.2983	accuracy[test]= 0.8955
Epoch 19:	loss[train]= 0.2644	accuracy[train]= 0.8920	loss[test]= 0.2980	accuracy[test]= 0.8943
Epoch 20:	loss[train]= 0.2638	accuracy[train]= 0.8920	loss[test]= 0.2976	accuracy[test]= 0.8955
Epoch 21:	loss[train]= 0.2633	accuracy[train]= 0.8932	loss[test]= 0.2972	accuracy[test]= 0.8955
Epoch 22:	loss[train]= 0.2629	accuracy[train]= 0.8955	loss[test]= 0.2968	accuracy[test]= 0.8966
Epoch 23:	loss[train]= 0.2625	accuracy[train]= 0.8966	loss[test]= 0.2965	accuracy[test]= 0.8966
Epoch 24:	loss[train]= 0.2621	accuracy[train]= 0.8955	loss[test]= 0.2961	accuracy[test]= 0.8966
Epoch 25:	loss[train]= 0.2617	accuracy[train]= 0.8955	loss[test]= 0.2957	accuracy[test]= 0.8966
Epoch 26:	loss[train]= 0.2614	accuracy[train]= 0.8943	loss[test]= 0.2953	accuracy[test]= 0.8966
Epoch 27:	loss[train]= 0.2610	accuracy[train]= 0.8920	loss[test]= 0.2949	accuracy[test]= 0.8966

Epoch 28: loss[train]= 0.2607 accuracy[train]= 0.8920 loss[test]= 0.2946 accuracy[test]= 0.8977
 Epoch 29: loss[train]= 0.2604 accuracy[train]= 0.8920 loss[test]= 0.2942 accuracy[test]= 0.8977



(g) How does the performance of this network compare with the previous networks. Can you explain this result? What is the influence of the activation functions in the network? (1 point)

TODO: Your answer here. : DONE

In this case, the behavior of having two linear layers without the activation function is almost the same as just having one linear layer - actually, mathematically it is equivalent.

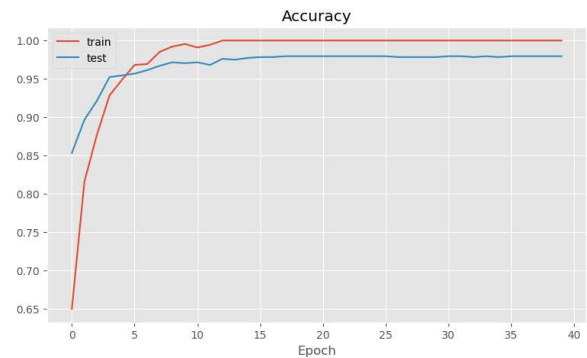
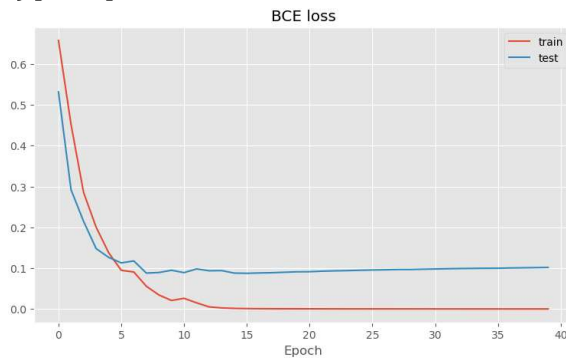
(h) One way to improve the performance of a neural network is by increasing the number of layers. Try a deeper network (e.g., a network with four linear layers) to see if this outperforms the previous networks. (1 point)

```
In [ ]: rng = np.random.default_rng(12345)
net = Net([
    Linear(64, 32),
    ReLU(),
    Linear(32, 16),
    ReLU(),
    Linear(16, 8),
    ReLU(),
    Linear(8, 1),
    Sigmoid()])

# TODO: tune the hyperparameters : DONE
fit(net, x, y,
    epochs = 40,
    learning_rate = 0.01,
    mb_size = 20)
```

Epoch 0:	loss[train]= 0.6575	accuracy[train]= 0.6500	loss[test]= 0.5317	accuracy[test]= 0.8534
Epoch 1:	loss[train]= 0.4520	accuracy[train]= 0.8159	loss[test]= 0.2921	accuracy[test]= 0.8966
Epoch 2:	loss[train]= 0.2854	accuracy[train]= 0.8773	loss[test]= 0.2147	accuracy[test]= 0.9216
Epoch 3:	loss[train]= 0.2004	accuracy[train]= 0.9284	loss[test]= 0.1479	accuracy[test]= 0.9523
Epoch 4:	loss[train]= 0.1376	accuracy[train]= 0.9489	loss[test]= 0.1263	accuracy[test]= 0.9545
Epoch 5:	loss[train]= 0.0947	accuracy[train]= 0.9682	loss[test]= 0.1130	accuracy[test]= 0.9568
Epoch 6:	loss[train]= 0.0907	accuracy[train]= 0.9693	loss[test]= 0.1177	accuracy[test]= 0.9614
Epoch 7:	loss[train]= 0.0556	accuracy[train]= 0.9852	loss[test]= 0.0880	accuracy[test]= 0.9670
Epoch 8:	loss[train]= 0.0344	accuracy[train]= 0.9920	loss[test]= 0.0894	accuracy[test]= 0.9716
Epoch 9:	loss[train]= 0.0209	accuracy[train]= 0.9955	loss[test]= 0.0949	accuracy[test]= 0.9705
Epoch 10:	loss[train]= 0.0262	accuracy[train]= 0.9909	loss[test]= 0.0891	accuracy[test]= 0.9716
Epoch 11:	loss[train]= 0.0151	accuracy[train]= 0.9943	loss[test]= 0.0982	accuracy[test]= 0.9682
Epoch 12:	loss[train]= 0.0052	accuracy[train]= 1.0000	loss[test]= 0.0937	accuracy[test]= 0.9761
Epoch 13:	loss[train]= 0.0026	accuracy[train]= 1.0000	loss[test]= 0.0941	accuracy[test]= 0.9750
Epoch 14:	loss[train]= 0.0016	accuracy[train]= 1.0000	loss[test]= 0.0880	accuracy[test]= 0.9773
Epoch 15:	loss[train]= 0.0010	accuracy[train]= 1.0000	loss[test]= 0.0874	accuracy[test]= 0.9784
Epoch 16:	loss[train]= 0.0008	accuracy[train]= 1.0000	loss[test]= 0.0883	accuracy[test]= 0.9784
Epoch 17:	loss[train]= 0.0007	accuracy[train]= 1.0000	loss[test]= 0.0889	accuracy[test]= 0.9795
Epoch 18:	loss[train]= 0.0006	accuracy[train]= 1.0000	loss[test]= 0.0900	accuracy[test]= 0.9795
Epoch 19:	loss[train]= 0.0005	accuracy[train]= 1.0000	loss[test]= 0.0911	accuracy[test]= 0.9795
Epoch 20:	loss[train]= 0.0004	accuracy[train]= 1.0000	loss[test]= 0.0913	accuracy[test]= 0.9795
Epoch 21:	loss[train]= 0.0004	accuracy[train]= 1.0000	loss[test]= 0.0927	accuracy[test]= 0.9795
Epoch 22:	loss[train]= 0.0004	accuracy[train]= 1.0000	loss[test]= 0.0935	accuracy[test]= 0.9795
Epoch 23:	loss[train]= 0.0003	accuracy[train]= 1.0000	loss[test]= 0.0941	accuracy[test]= 0.9795
Epoch 24:	loss[train]= 0.0003	accuracy[train]= 1.0000	loss[test]= 0.0949	accuracy[test]= 0.9795
Epoch 25:	loss[train]= 0.0003	accuracy[train]= 1.0000	loss[test]= 0.0955	accuracy[test]= 0.9795
Epoch 26:	loss[train]= 0.0003	accuracy[train]= 1.0000	loss[test]= 0.0961	accuracy[test]= 0.9784
Epoch 27:	loss[train]= 0.0002	accuracy[train]= 1.0000	loss[test]= 0.0967	accuracy[test]= 0.9784

Epoch 28: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0967 accuracy[test]= 0.9784
Epoch 29: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0974 accuracy[test]= 0.9784
Epoch 30: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0981 accuracy[test]= 0.9795
Epoch 31: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0986 accuracy[test]= 0.9795
Epoch 32: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0991 accuracy[test]= 0.9784
Epoch 33: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0994 accuracy[test]= 0.9795
Epoch 34: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0998 accuracy[test]= 0.9784
Epoch 35: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.0999 accuracy[test]= 0.9795
Epoch 36: loss[train]= 0.0002 accuracy[train]= 1.0000 loss[test]= 0.1006 accuracy[test]= 0.9795
Epoch 37: loss[train]= 0.0001 accuracy[train]= 1.0000 loss[test]= 0.1010 accuracy[test]= 0.9795
Epoch 38: loss[train]= 0.0001 accuracy[train]= 1.0000 loss[test]= 0.1014 accuracy[test]= 0.9795
Epoch 39: loss[train]= 0.0001 accuracy[train]= 1.0000 loss[test]= 0.1018 accuracy[test]= 0.9795



(i) Discuss your findings. Were you able to obtain a perfect classification? Explain the learning curves. (1 point)

TODO: Your answer here. : DONE

It is almost impossible to obtain the perfect classification, although the value of 0.978 is pretty close to 1. In this case, the addition of linear layers with their respective activation function has improved the obtained accuracy in comparison with just the linear network, but it does not significantly improve the 2-layer network obtained in b).

1.7 Final questions (6 points)

You now have some experience training neural networks. Time for a few final questions.

(a) What is the influence of the learning rate? What happens if the learning rate is too low or too high? (2 points)

TODO: Your answer here.: DONE

The learning rate determines the trade-off between training speed and stability. If it's too low, training may be slow, and if it's too high, it may lead to instability. Tuning the learning rate will contribute to achieve optimal performance.

(b) What is the role of the minibatch size in SGD? Explain the downsides of a minibatch size that is too small or too high. (2 points)

TODO: Your answer here. : DONE

The minibatch size impacts on training efficiency. Too small sizes lead to slow convergence and noisy gradients, while too large sizes may cause memory issues and poorer generalization.

(c) In the linear layer, we initialized the weights w with random values, but we initialized the bias b with zeros. What would happen if the weights w were initialised as zeros? Why is this not a problem for the bias? (2 points)

TODO: Your answer here.: DONE

When all weights are initialized to zero, all neurons in the layer would be computing the same output, and the network wouldn't learn effectively through backpropagation. Whereas setting the bias to zero gives the neuron its own choice, leading to the possibility of learning different aspects from the data.

The end

Well done! Please double check the instructions at the top before you submit your results.

This assignment has 45 points.

Version 00f98aa / 2023-09-04